//Assignment 2

Stack:

Implement stack as an abstract data type using singly linked list and use this ADT for conversion of infix expression to postfix, prefix

```
#include <iostream>
#include <stack>
#include <algorithm>
using namespace std;

// Node for Linked List
class Node
{
public:
   char data;
   Node *next;

   Node(char x)
   {
      data = x;
      next = NULL;
   }
};

// Stack ADT using Linked List
class Stack
{
   Node *top;

public:
   Stack()
   {
      top = NULL;
   }

   void push(char x)
   {
      Node *newNode = new Node(x);
      newNode->next = top;
      top = newNode;
   }

   char pop()
   {
```

```
        if (top == NULL)
            return '\0';

        char x = top->data;
        Node *temp = top;
        top = top->next;
        delete temp;
        return x;
    }

    char peek()
    {
        if (top == NULL)
            return '\0';
        return top->data;
    }

    bool isEmpty()
    {
        return top == NULL;
    }
};

// Function to check precedence
int precedence(char op)
{
    if (op == '^')
        return 3;
    if (op == '*' || op == '/')
        return 2;
    if (op == '+' || op == '-')
        return 1;
    return 0;
}

// Infix to Postfix
string infixToPostfix(string infix)
{
    Stack s;
    string postfix = "";

    for (int i = 0; i < infix.length(); i++)
    {
        char ch = infix[i];

        if (isalnum(ch))
```

```
        {
            postfix += ch;
        }
        else if (ch == '(')
        {
            s.push(ch);
        }
        else if (ch == ')')
        {
            while (!s.isEmpty() && s.peek() != '(')
                postfix += s.pop();

            s.pop(); // Remove '('
        }
        else
        {
            while (!s.isEmpty() &&
                   precedence(s.peek()) >= precedence(ch))
            {
                postfix += s.pop();
            }
            s.push(ch);
        }
    }

    while (!s.isEmpty())
        postfix += s.pop();

    return postfix;
}

// Infix to Prefix
string infixToPrefix(string infix)
{
    reverse(infix.begin(), infix.end());

    for (int i = 0; i < infix.length(); i++)
    {
        if (infix[i] == '(')
            infix[i] = ')';
        else if (infix[i] == ')')
            infix[i] = '(';
    }

    string postfix = infixToPostfix(infix);
```

```
    reverse(postfix.begin(), postfix.end());

    return postfix;
}

int main()
{
    string infix;

    cout << "Enter Infix Expression : ";
    cin >> infix;

    cout << "\nPostfix Expression : "
         << infixToPostfix(infix);

    cout << "\nPrefix Expression  : "
         << infixToPrefix(infix);

    return 0;
}
```